

PRD Pipeline — Proposed Infrastructure Changes

Internal proposal · 20 May 2026 · For sign-off before implementation

Two non-feature changes are recommended before further development. Both address silent-failure risks in the current production setup. Together: roughly one engineering day.

1. Replacing the Teams notification webhook

The current notification system uses Office 365 MessageCards, which Microsoft retired at the end of 2025. Existing webhook URLs may stop working without notice when our tenant is cut over. The five realistic replacement services are listed below.

OPTION A — POWER AUTOMATE WORKFLOWS (RECOMMENDED)

Service	Microsoft Power Automate, "Post to channel when a webhook request is received" template.
What it is	Microsoft's official replacement for the retired connector. Still posts into Teams channels using the rich Adaptive Card format.
Cost	Free — uses the Power Automate standard tier already included with Microsoft 365 licences.
Setup	One-time flow per Teams channel by an M365 admin. Copy the generated URL into our environment variables.
When to pick	Default choice. Minimum change to current workflow, supported by Microsoft long-term.

OPTION B — MICROSOFT GRAPH API

Service	Microsoft Graph (<i>graph.microsoft.com/v1.0/teams/{id}/channels/{id}/messages</i>).
What it is	Direct REST API call to post messages into Teams channels. Supports replies, @mentions, threading, edits.
Cost	Free — Graph API has no per-call fee under normal usage.
Setup	Register an Azure AD application; grant <i>ChannelMessage.Send</i> permission; configure OAuth2 client credentials.
When to pick	If we ever need richer features (replies, mentions, reactions). Heavier auth setup; overkill for current scope.

OPTION C — EMAIL VIA AWS SES

Service	Amazon Simple Email Service (already available in our AWS account).
What it is	Transactional email. Each input request becomes an email to the recipient with a deep link.
Cost	\$0.10 per 1,000 emails. Effectively free at our volume. Free tier covers 62,000 emails/month from EC2.

Setup	Verify our sending domain (DNS records); request SES production access from AWS (typically same-day approval).
When to pick	If we want notifications that don't depend on Teams at all, or as a fallback alongside Option A.

OPTION D — IN-APP NOTIFICATIONS ONLY (NO EXTERNAL SERVICE)

Service	None. The notification lives inside our own app.
What it is	A notification badge in the navigation bar that shows pending input requests when the user logs in. The data is already collected — only the UI is missing.
Cost	Zero. No third-party dependency.
Setup	Add a badge component and a small polling call to the existing endpoint.
When to pick	If push notifications aren't critical. Trade-off: recipients only see requests when they next visit the app.

OPTION E — SLACK INCOMING WEBHOOKS

Service	Slack incoming webhooks (still actively supported, unlike Teams connectors).
Cost	Free for any Slack workspace tier.
Setup	Create a Slack app, add an incoming webhook for the target channel, copy URL into our environment variables.
When to pick	Only relevant if Rian considers Slack as its primary chat platform. Not useful while we are on Teams.

RECOMMENDATION

Primary	Option A — Power Automate Workflows. Smallest change, stays on Teams, fully supported.
Optional add-on	Option D — In-app badge. Inexpensive and removes any "did the notification arrive?" doubt.
Effort	Small — about half a day for Option A; add 1–2 hours for Option D.
Risk if no action	Notifications silently stop. The Request Input collaboration feature breaks without raising any visible error.

2. Session storage — survive deploys and scaling

What we change	Move user-session storage from server memory to the existing Supabase Postgres database. Optionally extend auto-save coverage to a handful of pipeline stages that currently don't persist.
Why	Today, sessions live in the memory of a single EC2 instance. Every backend deploy or auto-scaling event silently logs all active users out and loses any work that hasn't yet hit auto-save.

CURRENT BEHAVIOUR

- Hosting auto-scales between 1 and 4 instances.
- Sessions only exist on the instance that created them.

- Project state auto-saves at some pipeline transitions, but not during the intake conversation or while the PRD is being streamed.

Code change	Activate a session store that is already partially coded in <i>server/services/sessionStore.js</i> ; add one environment variable; run one SQL migration on Supabase.
Optional add-on	Save pipeline state at the missing transition points (mid-intake, post-Writer, after each specialist agent). About 20 minutes of additional work.
Risk if no action	Users lose their place every time we deploy. Failure mode worsens if traffic ever causes auto-scaling to add a second instance — at that point, roughly half of every user's requests will fail with a re-login prompt.
Effort	Small — about one hour including the optional add-on. No new infrastructure or cost.